Risk & Compliance Dashboard — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Risk & Compliance Dashboard (Microsoft Purview) 4
What you get . . . 4
What this delivers . . . 4
The deployment model . . . 4
An honest note . . . 5
Outcome . . . 5
1. Solution Overview — Risk & Compliance Dashboard (Microsoft Purview) 5
What this solution is . . . 5
The consolidated dashboard in one paragraph . . . 5
The signals it consolidates (each named) . . . 6
What is authored exactly vs deferred . . . 6
Privacy-by-design (Insider Risk & Communication Compliance) . . . 7
Benefits . . . 7
Limitations (be honest with buyers) . . . 7
The standard deployment model . . . 7
2. Requirements Guide — Risk & Compliance Dashboard (Microsoft Purview) 8
2.1 Decide which Purview signals are in scope (and licensed) . . . 8
2.2 Least-privilege READ roles for each in-scope signal (role separation) . . . 8
2.3 Unified audit-log ingestion enabled (Audit) . . . 9
2.4 Security & Compliance PowerShell + Graph toolchain (category G3) . . . 9
2.5 Dashboard config: tiles, thresholds, cadence, distribution . . . 9
2.6 Owner-captured figures for signals without a reliable public read . . . 9
2.7 Compliance, records, privacy & legal stakeholders (category G5) . . . 10
2.8 Pilot scope + change control (category G6) . . . 10
Checklist . . . 10
Global Requirements — Microsoft Purview Solutions 10
1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11
2. Purview roles (least-privilege) and portal access . . . 11
3. Security & Compliance PowerShell + Graph toolchain . . . 11
4. Data sources & scope in scope . . . 12
5. Compliance, records, privacy & legal stakeholders . . . 12
6. A pilot scope and change control . . . 12
3. Implementation Guide — Risk & Compliance Dashboard (Microsoft Purview) 13
Phase 1 — Assess (what is licensed and readable) . . . 13
Phase 2 — Design (tiles, thresholds, cadence) . . . 14

Phase 3 — Configure & pilot (read-only / `-WhatIf`) . . . 14
Phase 4 — Validate (reconcile to source) . . . 15
Phase 5 — Expand (schedule, add signals, distribute) . . . 15
5. Testing Guide — Risk & Compliance Dashboard (Microsoft Purview) 16
Test 1 — Signal availability is reported honestly . . . 16
Test 2 — Role separation holds . . . 16
Test 3 — Unified audit-log ingestion state is correct . . . 16
Test 4 — Config validates (tiles, thresholds, sources) . . . 16
Test 5 — Collection writes a normalized dataset . . . 17
Test 6 — Dashboard renders with RAG and source badges . . . 17
Test 7 — HR-sensitive tiles are aggregate-only . . . 17
Test 8 — Live tiles reconcile against the portal . . . 17
Test 9 — Owner-captured figures are current . . . 17
Test 10 — Scheduled run reproduces the result . . . 17
Pass/fail gate . . . 17
6. Troubleshooting — Risk & Compliance Dashboard (Microsoft Purview) 18
Connectivity & licensing . . . 18
Roles & separation (CMPDASH-Scope.ps1) . . . 18
Audit ingestion & DLP volume . . . 19
Configuration (CMPDASH-Set-Configuration.ps1) . . . 19
Rendering & reconciliation . . . 19
Scheduling & distribution . . . 19
What this dashboard does NOT decide . . . 19
7. Best Practices — Risk & Compliance Dashboard (Microsoft Purview) 20
Design . . . 20
Sources & honesty . . . 20
Privacy & role separation (Insider Risk & Communication Compliance) . . . 20
Audit . . . 21
Operations . . . 21
Automation & security . . . 21
8. Deliverables Summary — Risk & Compliance Dashboard (Microsoft Purview) 22
Documentation . . . 22
PowerShell scripts (Security & Compliance PowerShell / Microsoft Graph) . . . 22
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Risk & Compliance Dashboard (Microsoft Purview)

Risk and compliance signals in Microsoft Purview are **scattered** across eight separate surfaces — Compliance Manager, Data Loss Prevention, sensitivity labels, retention & records, Insider Risk, Communication Compliance, Audit, and Data Security Posture Management. A leader who wants to answer "how is our compliance posture?" has to open eight cards and stitch the picture together by hand. Left that way, gaps hide, trends are invisible, and audit questions take days.

This solution delivers a **consolidated risk & compliance dashboard**: it **reads** each Purview signal that exposes a reliable public interface (via **Security & Compliance PowerShell** and **Microsoft Graph**), **captures** the ones that don't (via named owners, honestly flagged), and **renders** them in **one view** with red/amber/green status and volume trend — collected, validated against the portal, and refreshed on a schedule. It is a complete, **deploy-it-today** package written for an administrator with no prior Purview-reporting experience, and it stays **read-only** by default.

What you get

- A plain-language **Solution Overview** of the consolidated-dashboard model, the eight signals, and how

live reads and owner-captured figures are kept honestly separate.

- A **Requirements Guide** with verify / implement-if-missing for every prerequisite (the category

Purview requirements — compliance licensing, least-privilege roles, S&C PowerShell + Graph toolchain, stakeholders, pilot — are referenced, not repeated).

- An **Implementation Guide** of precise, ordered **S&C PowerShell + Graph + Purview portal** steps:

discover signals, author the config (tiles, RAG thresholds, cadence), pilot collection and rendering **read-only**, reconcile against the portal, then schedule and distribute — each artifact named, with pass/fail gates and rollback.

- Five **PowerShell scripts** for signal discovery, safe configuration (`-WhatIf`/`-Commit`), the

reconciliation validation, the consolidated dashboard generator (HTML + CSV with RAG and source badges), and a scope/health helper that checks role separation.

- **Testing** and **Troubleshooting** guides for a confident rollout.

What this delivers

One view of compliance and data risk across Microsoft Purview — configured, piloted read-only, reconciled against the source, and refreshed on a schedule — that a compliance lead, CISO, or auditor can trust.

The deployment model

Assess → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand** — assess what's licensed and readable, design the tiles and thresholds, configure collection and rendering, pilot read-only,

validate the numbers, then schedule and distribute.

An honest note

This is a **reporting layer, not a control** — it reflects posture; it does not enforce policy or make **regulatory, records, or legal determinations**, which remain with the governance functions. Several signals require **E5 / E5 Compliance**, and a few (Compliance Manager score, Insider Risk / Communication Compliance detail, DSPM) have **no reliable public read** — those are **owner-captured** and flagged **"verify your licensing/tenant."** The HR-sensitive feeds are shown as **aggregate counts only**, with strict **role separation**.

Outcome

A consolidated, honest, privacy-respecting risk & compliance dashboard across Microsoft Purview signals — validated against the portal and refreshed on a schedule — so posture is visible, trends are trackable, and the capability is defensible.

1. Solution Overview — Risk & Compliance Dashboard (Microsoft Purview)

What this solution is

This solution builds a **consolidated risk & compliance dashboard** across the signals Microsoft Purview already produces — **Compliance Manager** score, **Data Loss Prevention (DLP)**, **sensitivity labels**, **data lifecycle & records (retention)**, **Insider Risk Management**, **Communication Compliance**, **Audit**, and **Data Security Posture Management (DSPM)** — and presents them in **one view** with red/amber/green (RAG) status, coverage, and volume trend. Instead of chasing eight separate Purview surfaces, a compliance lead, CISO, or auditor sees the whole posture in a single, repeatable report that is **collected, rendered, validated, and refreshed** on a schedule.

Purview does **not** ship a single native "risk & compliance dashboard" that spans every one of these workloads — each capability has its own portal card and its own report. This solution's job is to **aggregate** those existing signals: it reads each feed (via **Security & Compliance PowerShell** and **Microsoft Graph** where a reliable public interface exists, and via **owner-captured portal figures** where one does not), normalizes them, and renders a consolidated dashboard artifact you own and control.

It is **vendor-neutral / white-label** and written for an administrator with no prior Purview-reporting experience. It reads signals **read-only**; the only write action (enabling **unified audit-log ingestion** if it is off, and writing the dashboard config/schedule) is gated behind an explicit `-Commit` flag and piloted first.

The consolidated dashboard in one paragraph

The dashboard is a **derived reporting layer**, not a new policy engine. A **collection** run reads each in-scope Purview signal and writes a normalized dataset (`RiskComplianceDashboard.data.json`); a **render** step turns that dataset into a single **HTML dashboard** (`RiskComplianceDashboard.html`) plus a **CSV** summary, each metric shown with a **RAG threshold** defined in the dashboard config (`RiskComplianceDashboard.config.json`). Because it is derived, it never changes a policy, never

makes a regulatory or records determination, and — critically for the HR-sensitive feeds — shows Insider Risk and Communication Compliance as **aggregate counts only**, never case content.

The signals it consolidates (each named)

Each signal is a **tile** on the dashboard. Availability is **licensing-dependent** — many require **Microsoft 365 E5** or the **E5 Compliance** add-on, and some are in preview — so every tile records *whether the capability is licensed/enabled in your tenant* and is flagged **"verify your licensing/tenant"** where availability or interface is not guaranteed.

1. **Compliance Manager — compliance score.** Overall score and top improvement actions. *(A reliable

public read interface is limited; captured from the portal by the owner where no cmdlet exists — flagged "verify your licensing/tenant".)*

1. **Data Loss Prevention (DLP).** Policy count, **enforced vs simulation/test** split, and **match /

incident volume** over the window (from the unified audit log). High-value tile because DLP posture is the most common audit question.

1. **Sensitivity labels.** Published labels, label-policy coverage, and labeling activity volume.

2. **Data lifecycle & records (retention).** Retention policy count and **workload coverage** (Exchange /

SharePoint / OneDrive / Teams).

1. **Insider Risk Management.** **Active policy count and alert counts — aggregate only, pseudonymized,

role-separated. **No case content in this dashboard;** case actions and determinations are deferred to HR/legal.** *(E5; public read interface limited — aggregate figures owner-captured where no cmdlet.)*

1. **Communication Compliance.** Policy count and pending-item counts — **aggregate only, role-separated;

review outcomes deferred to HR/legal.** *(E5.)*

1. **Audit.** Whether **unified audit-log ingestion** is on, **Audit Standard vs Premium**, and retention.

Audit underpins the DLP/label volume tiles, so its health is shown first.

1. **Data Security Posture Management (DSPM).** Posture recommendations / risk indicators. *(E5; interface

evolving — owner-captured / portal where no reliable public read — flagged "verify your licensing/tenant".)*

What is authored exactly vs deferred

- **Exact detail is given** for the **Microsoft Purview portal** (`purview.microsoft.com`), **Security &

Compliance PowerShell (`Connect-IPPSSession`), and** Microsoft Graph** — for the feeds that have a reliable public interface (DLP/label/retention policy enumeration, unified audit-log queries, audit ingestion state).

- **Signals without a reliable, current public read interface** (Compliance Manager score, Insider Risk /

Communication Compliance case detail, DSPM posture) are **not invented** — the dashboard captures them as **owner-entered figures** in the config, each flagged **"verify your licensing/tenant"**, so the consolidated view is honest about its sources.

- **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, insider-risk / communication-compliance case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — this solution provides the **method and the evidence**, not the determination.

Privacy-by-design (Insider Risk & Communication Compliance)

These feeds are **HR- and privacy-sensitive**. On this dashboard they appear **only as aggregate counts** (e.g. "policies active: 4; open alerts: 12"), **never** as user names, case content, or reviewable items. **Role separation is enforced**: the account that builds the dashboard needs only the least-privilege **read** roles for the metrics it consolidates, and the Insider Risk / Communication Compliance figures are **entered by the authorized owner of those programs**, not scraped, so the dashboard cannot become a back-door into case data.

Benefits

- **One view** — eight Purview signals in a single RAG dashboard, refreshed on a schedule.
- **Owned and portable** — a local HTML/CSV artifact you control; no dependency on a signal you can't read.
- **Honest about sources** — every tile shows whether it's a live read or an owner-captured figure, and whether the capability is licensed.
- **Safe** — read-only by default; the one write action is `-Commit`-gated and piloted first.
- **Privacy-respecting** — HR-sensitive feeds are aggregate-only, role-separated, determinations deferred.

Limitations (be honest with buyers)

- **This is a reporting layer, not a control.** It reflects posture; it does not enforce DLP, apply labels, or make retention/records decisions. Those live in their own Purview solutions.
- **Licensing gates the tiles.** Several signals require **E5 / E5 Compliance**; unlicensed tiles show "not licensed / verify your tenant" rather than fabricated numbers.
- **Some feeds have no reliable public read.** Compliance Manager score, Insider Risk / Communication Compliance detail, and DSPM are **owner-captured** where Microsoft exposes no stable public cmdlet — **verify your licensing/tenant**; the portal remains the system of record.
- **No determinations.** Regulatory, records, and legal calls are deferred to the governance functions.

The standard deployment model

> ***Assess which signals are licensed and readable → design the dashboard (tiles, thresholds, cadence, distribution) → configure collection + rendering (pilot in read-only /*** `-WhatIf`***) → validate the numbers against the portal → expand (schedule, add signals, distribute).*** *Pattern:* ***Assess → Design → Configure → Pilot (simulate/test) → Validate → Expand****, with explicit pass/fail gates and a documented rollback.*

2. Requirements Guide — Risk & Compliance Dashboard (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide Purview prerequisites (a tenant with the required **compliance licensing**, least-privilege **Purview roles**, the **Security & Compliance PowerShell + Graph** toolchain, **in-scope data sources**, **compliance/records/privacy/legal stakeholders**, and a **pilot scope + change control**) are in `global-requirements-purview.md` — confirm those **first**; this guide adds the dashboard-specific prerequisites and does not duplicate the six.

> ***Object names used throughout (use verbatim):*** *dashboard config* `RiskComplianceDashboard.config.json`*; collected dataset* `RiskComplianceDashboard.data.json`*; rendered artifacts* `RiskComplianceDashboard.html` + `RiskComplianceDashboard.csv`*; signal-scope label* ***"Risk & Compliance Dashboard — Signal Scope"****; scripts* `CMPDASH-Discovery.ps1`*,* `CMPDASH-Set-Configuration.ps1`*,* `CMPDASH-Validation.ps1`*,* `CMPDASH-Report.ps1`*,* `CMPDASH-Scope.ps1`*.*

2.1 Decide which Purview signals are in scope (and licensed)

Description: A confirmed list of the signals the dashboard will consolidate — from the eight candidates: **Compliance Manager, DLP, sensitivity labels, retention/records, Insider Risk, Communication Compliance, Audit, DSPM** — and, for each, whether it is **licensed and enabled** in your tenant. Baseline signals (DLP/label/retention enumeration, Audit) need **E3+**; advanced signals (Insider Risk, Communication Compliance, DSPM, Audit Premium, some Compliance Manager templates) need **E5 / E5 Compliance**. **Why required:** The dashboard only shows what the tenant licenses and exposes; an unlicensed signal is a "not licensed" tile, not a number to fabricate. **How to verify:** Run `CMPDASH-Discovery.ps1` (reports which signals are present/enabled and which require E5); cross-check the subscription in the M365 admin center (category G1). **Verify your licensing/tenant** for any signal the script flags as uncertain. **How to implement if missing:** Assign/purchase the licensing for the signals you require, or scope the dashboard to the signals you have.

2.2 Least-privilege READ roles for each in-scope signal (role separation)

Description: The account that collects the dashboard holds only the **least-privilege read roles** needed: **Compliance Administrator** or **Global Reader** for cross-signal read; **Compliance Data Administrator** / **Information Protection Reader** for DLP/label/retention enumeration; **View-Only Audit Logs** (or the **Audit Reader** role) for the audit queries. **Insider Risk** and **Communication Compliance** figures are **owner-captured by the authorized program owners of those roles — the dashboard account is NOT granted Insider Risk / Communication Compliance investigator roles** (strict role separation; HR-sensitive). **Why required:** Purview uses fine-grained role groups; the collector needs read on each feed but must **not** accumulate investigative access to HR-sensitive case data. **How to verify:** Purview

portal → **Roles & scopes / Permissions** confirms the read roles; confirm the dashboard account is **not** in Insider Risk / Communication Compliance investigator role groups. **How to implement if missing:** Assign the least-privilege read roles under change control; keep the HR-sensitive investigator roles with their program owners.

2.3 Unified audit-log ingestion enabled (Audit)

Description: Unified audit-log ingestion is turned on so DLP-match, labeling, and other activity volumes are queryable via `Search-UnifiedAuditLog`. Confirm **Audit Standard vs Premium** and the **retention** that applies (Premium extends retention; some record types need E5). **Why required:** The DLP-volume and labeling-activity tiles read the unified audit log; if ingestion is off, those tiles have no source. **How to verify:** `CMPDASH-Discovery.ps1` reports the ingestion state (from `Get-AdminAuditLogConfig`); Purview portal → **Audit** shows search is available. **How to implement if missing:** Enable ingestion — `CMPDASH-Set-Configuration.ps1 -Commit` can turn it on (gated), or set it in the portal. Note audit begins capturing from when it is enabled; historical volume before that date is not available.

2.4 Security & Compliance PowerShell + Graph toolchain (category G3)

Description: A workstation with **ExchangeOnlineManagement** (`Connect-IPPSSession`) and **Microsoft Graph PowerShell**, signed in as a modern-auth / MFA-capable admin with the read roles from 2.2. **Why required:** Collection, validation, and rendering run through these; the scripts call `Connect-IPPSSession` and/or `Connect-MgGraph`. **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed; `CMPDASH-Scope.ps1` reports which feeds the current identity can read. **How to implement if missing:** Install the modules and confirm modern-auth sign-in (category G3).

2.5 Dashboard config: tiles, thresholds, cadence, distribution

Description: A **dashboard config** (`RiskComplianceDashboard.config.json`) defining, per signal: the **metric**, the **RAG thresholds** (e.g. compliance score green ≥ 80, amber 60–79, red < 60 — thresholds are **your** policy, not a Microsoft default), the **refresh cadence**, the **owner-captured fields** (for signals with no live read), and the **distribution list** for the rendered dashboard. **Why required:** RAG status and cadence are organizational decisions; the render step and validation both key off this config. **How to verify:** The config exists and validates; `CMPDASH-Set-Configuration.ps1 -WhatIf` shows the tiles/thresholds it would apply. **How to implement if missing:** Create it with `CMPDASH-Set-Configuration.ps1` (default `-WhatIf`; `-Commit` writes it) and confirm thresholds with your compliance stakeholders.

2.6 Owner-captured figures for signals without a reliable public read

Description: Named **owners** and a capture cadence for the signals the dashboard **cannot read via a stable public interface** — **Compliance Manager score**, **Insider Risk** aggregate counts, **Communication Compliance** aggregate counts, and **DSPM** posture — where Microsoft exposes no reliable public cmdlet at authoring time. **Why required:** These tiles must be honest: an owner enters the current figure (from the portal) rather than the dashboard inventing one; the HR-sensitive figures **must** be entered by the program owner, not the collector (role separation). **How to verify:** The config's owner-captured fields have named owners and a "captured on" date; `CMPDASH-Validation.ps1` flags stale owner-captured figures. **Verify your licensing/tenant** — if Microsoft has since published a stable read interface for one of these, prefer it. **How to implement if missing:** Assign owners and a capture

cadence before the first published dashboard.

2.7 Compliance, records, privacy & legal stakeholders (category G5)

Description: The **compliance, records, privacy, security, and legal** stakeholders who define what "good" looks like (RAG thresholds), who receives the dashboard, and who owns each signal — and who make the **determinations** the dashboard never makes. **Why required:** Thresholds and the meaning of "red" are policy decisions; HR-sensitive feeds and any regulatory/records interpretation belong to these functions. **How to verify:** Named threshold approvers, dashboard recipients, and signal owners exist. **How to implement if missing:** Identify the owners and agree thresholds/distribution before publishing.

2.8 Pilot scope + change control (category G6)

Description: A **pilot** run of the dashboard (read-only collection + render, reconciled against the portal) before it is scheduled and distributed, and a **change-control** window for the one write action (enabling audit ingestion / writing the config/schedule). **Why required:** The dashboard must be **reconciled to the source** before anyone trusts it for decisions; enabling audit ingestion is tenant-wide and high-impact. **How to verify:** A named pilot window and approver exist; the first run is validated (Testing Guide). **How to implement if missing:** Agree the pilot and change control before scheduling or distributing.

Checklist

- In-scope signals decided; each confirmed licensed/enabled (2.1 / category G1)
- Least-privilege READ roles assigned; collector NOT in Insider Risk / Comm Compliance investigator roles (2.2)
- Unified audit-log ingestion on; Standard vs Premium + retention noted (2.3)
- `Connect-IPPSSession` + `Connect-MgGraph` succeed with the read roles (2.4 / category G3)
- `RiskComplianceDashboard.config.json` created: tiles, RAG thresholds, cadence, distribution (2.5)
- Owners + capture cadence set for owner-captured signals (Compliance Manager, Insider Risk, Comm Compliance, DSPM) (2.6)
- Threshold approvers, recipients, and signal owners named (2.7 / category G5)
- Pilot window + change control agreed for the `-Commit` write (2.8 / category G6)

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet*

*detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Risk & Compliance Dashboard (Microsoft Purview)

Follow these steps **in order**. **Security & Compliance PowerShell** (`Connect-IPPSSession`), **Microsoft Graph**, and **Microsoft Purview portal** (`purview.microsoft.com`) steps are given cmdlet- / click-exact **for the feeds that have a reliable public interface**. Signals with **no stable public read** (Compliance Manager score, Insider Risk / Communication Compliance detail, DSPM) are **owner-captured** — named and deferred, never invented. The category prerequisites in `global-requirements-purview.md` (licensing, roles, toolchain, stakeholders, pilot) are assumed in place.

Consoles / interfaces used:

- **Microsoft Purview portal** — `https://purview.microsoft.com`
- **Security & Compliance PowerShell** — `Connect-IPPSSession` (ExchangeOnlineManagement module)
- **Microsoft Graph** — `Connect-MgGraph` (read scopes only)

> ***Model:*** *Assess which signals are licensed/readable → design the tiles/thresholds/cadence → configure collection + rendering and pilot it* ***read-only /*** `-WhatIf` *→ validate the numbers against the portal → expand (schedule, add signals, distribute).* ***Assess → Design → Configure → Pilot (simulate/test) → Validate → Expand.***
>
> ***Artifact names (use verbatim):*** *config* `RiskComplianceDashboard.config.json`*; dataset* `RiskComplianceDashboard.data.json`*; rendered* `RiskComplianceDashboard.html` + `RiskComplianceDashboard.csv`*; signal-scope label* ***"Risk & Compliance Dashboard — Signal Scope"****.*
>
> ***Risk + rollback (read first):*** *The dashboard is a* ***read-only reporting layer*** *— collection, validation, and rendering* ***change no policy*** *and make* ***no determination****. The* ***only*** *write action is in* `CMPDASH-Set-Configuration.ps1` *and is* `-Commit`***-gated****: (a) writing the local config/schedule, and (b) optionally enabling* ***unified audit-log ingestion*** *(tenant-wide, high-impact — pilot + change control).* ***Everything defaults to*** `-WhatIf` ***/ read-only. Rollback:*** *delete the local config/data/ HTML artifacts to remove the dashboard (no tenant change); if audit ingestion was enabled by* `-Commit` *and you must revert, disable it in the portal / with* `Set-AdminAuditLogConfig` *under change control (note: disabling audit reduces your evidence — confirm with compliance first).* ***Insider Risk / Communication Compliance are aggregate-only and owner-captured — the dashboard never reads or exposes case content.***

Phase 1 — Assess (what is licensed and readable)

Step 1 — Discover the available signals and audit state

1. Connect read-only: `Connect-IPPSSession` and `Connect-MgGraph -Scopes 'AuditLog.Read.All','SecurityEvents.Read.All'`.
2. Run `CMPDASH-Discovery.ps1`. It reports, per signal: **present/enabled**, whether it requires

E5, the **DLP / label / retention policy counts**, and the **unified audit-log ingestion** state.

1. Run `CMPDASH-Scope.ps1` to confirm **which feeds the current identity can actually read** (role check) and each feed's **freshness/health**. Note any signal flagged **"verify your licensing/tenant"** — those become **owner-captured** tiles (Step 4), not live reads.

Step 2 — Confirm role separation

1. Confirm the collector identity holds only least-privilege **read** roles (Requirement 2.2) and is **not** a member of **Insider Risk Management** or **Communication Compliance** investigator role groups. `CMPDASH-Scope.ps1` reports the identity's readable feeds; the HR-sensitive feeds should show **"aggregate/owner-captured — not read by this account."**

Phase 2 — Design (tiles, thresholds, cadence)

Step 3 — Author the dashboard config

1. Create `RiskComplianceDashboard.config.json` with `CMPDASH-Set-Configuration.ps1 -WhatIf` (preview) — it emits the config scaffold. For each in-scope signal set:

- the **metric** (e.g. DLP: enforced-policy count + match volume over `-WindowDays`),
- the **RAG thresholds** (**your** policy — e.g. compliance score green ≥ 80 / amber 60–79 / red < 60; DLP simulation-only policies amber; audit ingestion off = red),
- the **source** (`live` for read feeds, `owner-captured` for Compliance Manager / Insider Risk / Communication Compliance / DSPM),
- the **refresh cadence** and the **distribution list**.

1. Confirm thresholds with your compliance stakeholders (Requirement 2.7) — the numbers are policy, not a Microsoft default.

Step 4 — Assign owners for owner-captured signals

1. In the config, set the **owner** and **capture cadence** for each `owner-captured` signal (Compliance Manager score; Insider Risk aggregate counts; Communication Compliance aggregate counts; DSPM posture). The **HR-sensitive figures must be entered by the program owner**, not the collector.

1. Record a **"captured on"** date per field so `CMPDASH-Validation.ps1` can flag stale figures.

Phase 3 — Configure & pilot (read-only / `-WhatIf`)

Step 5 — (If needed) enable unified audit-log ingestion — `-Commit`, piloted

1. If Step 1 showed ingestion **off** and you need the DLP/label volume tiles, enable it under change

control: `CMPDASH-Set-Configuration.ps1 -EnableAuditIngestion -WhatIf` to preview, then `-EnableAuditIngestion -Commit` to apply (calls `Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`).

1. Audit begins capturing **from now**; historical volume before this date is not available. If ingestion

is already on, skip this step.

Step 6 — Write the config/schedule — `-Commit`, piloted

1. Preview with `CMPDASH-Set-Configuration.ps1 -WhatIf`; when the tiles/thresholds are agreed, write

the config with `CMPDASH-Set-Configuration.ps1 -Commit`. This writes **only local files** (the config and, if requested, a scheduled-task scaffold) — it changes **no** Purview policy.

Step 7 — Pilot collection + render (read-only)

1. Run `CMPDASH-Report.ps1` (read-only). It **collects** every `live` signal (DLP/label/retention

enumeration + `Search-UnifiedAuditLog` volume over `-WindowDays`), merges the **owner-captured** figures from the config, writes the normalized `RiskComplianceDashboard.data.json`, and renders `RiskComplianceDashboard.html` + `RiskComplianceDashboard.csv` with RAG tiles.

1. Open the HTML dashboard. Every tile should show its **value, RAG status, and source badge**

(`live` vs `owner-captured`), and any unlicensed/unreadable signal should read **"not licensed / verify your tenant"** — never a fabricated number.

Phase 4 — Validate (reconcile to source)

Step 8 — Reconcile the dashboard against the portal (pilot gate)

1. Run `CMPDASH-Validation.ps1`. It re-reads each `live` signal and **reconciles** it against the

value in `RiskComplianceDashboard.data.json` (policy counts match; DLP volume within tolerance; audit-ingestion state correct), and **flags stale owner-captured figures** past their cadence.

1. Manually spot-check a **live** tile against the portal (e.g. DLP policy count in Purview → **Data loss

prevention → Policies **equals the dashboard's), and confirm each** owner-captured **tile matches the figure the program owner recorded. Complete the** Testing Guide** pilot gate before scheduling.

Phase 5 — Expand (schedule, add signals, distribute)

Step 9 — Schedule the refresh

1. Schedule `CMPDASH-Report.ps1` at the cadence in the config (e.g. Windows Task Scheduler / a runbook)

under a **least-privilege service identity** with the read roles from 2.2 — **no secrets in the script**; use an interactive modern-auth or managed identity per your standard.

1. Re-run `CMPDASH-Validation.ps1` after the first scheduled run to confirm the automated numbers still

reconcile.

Step 10 — Distribute and add signals in waves

1. Distribute `RiskComplianceDashboard.html` to the recipients in the config (e.g. a compliance SharePoint page or an emailed artifact). Do **not** distribute the raw dataset if any owner-captured figure is sensitive beyond the intended audience.
1. Add further signals in **waves** — start with the `live` feeds you can reconcile, add `owner-captured` tiles as owners commit to a cadence. Re-validate after each addition. **Verify your licensing/tenant** before promising a tile whose interface is flagged uncertain.

Order of operations

```
Discover signals + audit state → confirm role separation → author config (tiles/thresholds/cadence) → assign owner-captured owners → (enable audit ingestion, -Commit) → write config (-Commit) → pilot collect + render (read-only) → reconcile against portal (gate) → schedule → distribute + add signals in waves
```

5. Testing Guide — Risk & Compliance Dashboard (Microsoft Purview)

Run after the config is written and a pilot collection/render has completed (Implementation Steps 6–8), and again after each scheduled run or added signal. Back each test with `CMPDASH-Discovery.ps1` (readiness), `CMPDASH-Scope.ps1` (role/feed health), `CMPDASH-Report.ps1` (collect + render), and `CMPDASH-Validation.ps1` (reconcile). All are read-only except the `-Commit` config write.

Test 1 — Signal availability is reported honestly

Do: Run `CMPDASH-Discovery.ps1`. **Expected:** Each of the eight signals is reported **present/enabled** or **not licensed / verify your tenant**; E5-gated signals are flagged. No signal is reported as a number the tenant can't produce.

Test 2 — Role separation holds

Do: Run `CMPDASH-Scope.ps1` as the collector identity. **Expected:** The identity can read the **live** feeds (DLP/label/retention/audit) and is **NOT** shown with Insider Risk / Communication Compliance investigator access; those feeds report **"aggregate/owner-captured — not read by this account."**

Test 3 — Unified audit-log ingestion state is correct

Do: Run `CMPDASH-Discovery.ps1`; check the Audit tile. **Expected:** Ingestion **on** (or, if just enabled by `-Commit`, shown enabled with a "capturing from <date>" note); Standard vs Premium and retention reported. If off, the DLP/label volume tiles correctly show "no audit source."

Test 4 — Config validates (tiles, thresholds, sources)

Do: Run `CMPDASH-Set-Configuration.ps1 -WhatIf`. **Expected:** `RiskComplianceDashboard.config.json` parses; every in-scope signal has a metric, RAG thresholds, a **source** (`live` / `owner-captured`), a cadence, and (for owner-captured) a named owner.

Test 5 — Collection writes a normalized dataset

Do: Run `CMPDASH-Report.ps1`. **Expected:** `RiskComplianceDashboard.data.json` is written with one record per in-scope signal — value, RAG status, **source badge**, and timestamp; live feeds populated from Graph / S&C PowerShell, owner-captured feeds populated from the config.

Test 6 — Dashboard renders with RAG and source badges

Do: Open `RiskComplianceDashboard.html`. **Expected:** One tile per signal with **value + RAG colour + source badge** (`live` vs `owner-captured`); unlicensed/unreadable signals read **"not licensed / verify your tenant"**; the CSV mirrors the tiles.

Test 7 — HR-sensitive tiles are aggregate-only

Do: Inspect the Insider Risk and Communication Compliance tiles in the HTML and the dataset. **Expected: Counts only** (e.g. policies active, open alerts) — **no** user names, case content, or reviewable items anywhere in the artifacts. Source badge = `owner-captured`.

Test 8 — Live tiles reconcile against the portal

Do: Run `CMPDASH-Validation.ps1`; spot-check one live tile (e.g. DLP policy count) against Purview → **Data loss prevention** → **Policies**. **Expected:** The dashboard's live values **match** the portal within tolerance; validation reports **PASS** for each reconciled live signal, with any mismatch listed to fix.

Test 9 — Owner-captured figures are current

Do: Run `CMPDASH-Validation.ps1`. **Expected:** Each owner-captured tile has a **"captured on"** date within its cadence; any figure past cadence is flagged **STALE** with the owner to chase. No owner-captured figure is silently defaulted.

Test 10 — Scheduled run reproduces the result

Do: Trigger the scheduled `CMPDASH-Report.ps1`, then run `CMPDASH-Validation.ps1`. **Expected:** The automated run produces the dataset + HTML/CSV under the service identity, and the numbers still reconcile — proving the dashboard is repeatable, not a one-off.

Pass/fail gate

Gate	Requirement
Readiness pass	Tests 1–3 pass (signals reported honestly; role separation holds; audit state correct)

Gate	Requirement
Build pass	Tests 4–7 pass (config validates; dataset written; dashboard renders with source badges; HR tiles aggregate-only)
Trust pass	Tests 8–10 pass (live tiles reconcile; owner-captured figures current; scheduled run reproduces)

> *Do not distribute the dashboard for decisions until all three gates pass. Re-run Tests 5–10 after each scheduled run or added signal. **Regulatory/records/legal determinations remain deferred** — the dashboard evidences posture; it does not decide it.*

6. Troubleshooting — Risk & Compliance Dashboard (Microsoft Purview)

Common issues, grouped by stage. Each has a **symptom**, the **likely cause**, and a **resolution**. The dashboard is a **read-only reporting layer**: `CMPDASH-Discovery.ps1`, `CMPDASH-Scope.ps1`, `CMPDASH-Report.ps1`, and `CMPDASH-Validation.ps1` change no policy; the only write is `CMPDASH-Set-Configuration.ps1`, which is `-WhatIf` by default and (with `-Commit`) writes local config plus, only if you ask with `-EnableAuditIngestion`, the one tenant-wide audit setting. Most issues here are licensing, role, config, or freshness — not damage. A signal reading **"not licensed / verify your tenant"** or an empty feed reading **0** is a legitimate value, not a failure.

Connectivity & licensing

Symptom: live tiles all read "not licensed / verify your tenant". **Likely cause:** no active `Connect-IPPSSession`, or the reader role is missing. **Resolution:** connect as the least-privilege collector identity (Requirement 2.2) and re-run `CMPDASH-Scope.ps1` to confirm each feed reads OK before rendering.

Symptom: a signal you expected is owner-captured, not live. **Likely cause:** Compliance Manager score, Insider Risk, Communication Compliance, and DSPM have **no stable public read** — by design they are owner-captured. **Resolution:** set the `owner` + `capturedOn` fields in `RiskComplianceDashboard.config.json`; the program owner enters the aggregate figure. Never fabricate a live number for these.

Roles & separation (CMPDASH-Scope.ps1)

Symptom: `CMPDASH-Scope.ps1` reports "role separation is BROKEN". **Likely cause:** the collector identity is a member of an Insider Risk / Communication Compliance investigator role group and can read HR-sensitive interfaces. **Resolution:** remove the collector from those investigator role groups before piloting — the collector must read only least-privilege aggregate/live feeds. HR-sensitive figures are aggregate-only and owner-captured.

Symptom: a live feed shows "read FAILED — likely role/permission". **Likely cause:** the collector lacks the reader role for that feed. **Resolution:** grant the least-privilege reader (e.g. Global Reader / View-Only Audit Logs / Compliance Administrator as appropriate), wait for propagation, and re-run. Do not record the

signal as missing until the read succeeds.

Audit ingestion & DLP volume

Symptom: the DLP tile shows volume `n/a (audit ingestion off or no permission)`. **Likely cause:** unified audit-log ingestion is disabled, or the account can't run `Search-UnifiedAuditLog`. **Resolution:** enable ingestion under change control — `CMPDASH-Set-Configuration.ps1 -EnableAuditIngestion -WhatIf` to preview, then `-Commit`. Audit captures **from enablement forward**; historical volume before that date is not available.

Symptom: DLP volume is lower than expected. **Likely cause:** the report samples one record type (`ComplianceDLPExchange`) and a bounded `ResultSize`. **Resolution:** this is a known scoping note — extend the collection per workload if you need multi-workload volume; the policy counts (not the sampled volume) are the reconciled figure.

Configuration (CMPDASH-Set-Configuration.ps1)

Symptom: nothing was written. **Likely cause:** `-Commit` was omitted (default is `-WhatIf`). **Resolution:** re-run with `-Commit`; `RiskComplianceDashboard.config.json` is written locally (no Purview policy is changed).

Symptom: the RAG colours look wrong. **Likely cause:** the thresholds are **your** policy, not a Microsoft default, and haven't been agreed. **Resolution:** confirm the RAG thresholds with your compliance stakeholders (Requirement 2.7) and edit the config's `rag` blocks accordingly.

Rendering & reconciliation

Symptom: `CMPDASH-Report.ps1` says config not found. **Likely cause:** you rendered before `CMPDASH-Set-Configuration.ps1 -Commit` wrote the config. **Resolution:** write the config first, then render.

Symptom: `CMPDASH-Validation.ps1` reports FAIL for a live tile. **Likely cause:** the tenant state changed since the last render, or the dataset is stale. **Resolution:** re-run `CMPDASH-Report.ps1` to refresh `RiskComplianceDashboard.data.json`, then re-validate. Spot-check the tile against the Purview portal (e.g. DLP policy count in **Data loss prevention** → **Policies**).

Symptom: `CMPDASH-Validation.ps1` reports STALE for an owner-captured tile. **Likely cause:** the figure is older than `ownerCaptureMaxAgeDays`, or was never captured. **Resolution:** the program owner re-records the aggregate figure and updates `capturedOn`, then re-validate before scheduling/distributing.

Scheduling & distribution

Symptom: the scheduled run has no data / auth fails. **Likely cause:** the service identity lacks the read roles, or a secret was (incorrectly) expected in the script. **Resolution:** run under a least-privilege modern-auth or managed identity with the Requirement-2.2 read roles — **no secrets in the script**. Re-run `CMPDASH-Validation.ps1` after the first scheduled run.

What this dashboard does NOT decide

The dashboard **reports** posture — it makes **no determination**. Whether a score, a policy state, or an aggregate count is *acceptable* or *satisfies a regulation* is a **compliance / records / legal determination**. Insider Risk and Communication Compliance figures are **aggregate-only, owner-captured, and role-separated** — the dashboard never reads or exposes case content, and case outcomes are deferred to HR/legal.

7. Best Practices — Risk & Compliance Dashboard (Microsoft Purview)

Field-proven recommendations for a trustworthy, low-maintenance consolidated dashboard.

Design

- **Treat it as a reporting layer, not a control.** The dashboard reflects posture; it never enforces DLP, applies labels, or makes a retention/records call. Keep those decisions in their own Purview solutions and their governance owners — the dashboard's credibility depends on it staying read-only.
- **Thresholds are policy, not defaults.** RAG bands (what counts as red) are an organizational decision — agree them with compliance/records/legal (Requirement 2.7) and record them in the config, not in someone's head. Revisit them as posture matures.
- **One tile per signal, with an honest source badge.** Every tile shows whether it's a **live** read or an **owner-captured** figure. Never blur the two; a fabricated number destroys trust in the whole board.
- **Start with what you can reconcile.** Ship the `live` feeds you can validate against the portal first (DLP/label/retention/audit); add `owner-captured` tiles as owners commit to a cadence.

Sources & honesty

- **Never invent a feed.** Where Microsoft exposes no reliable public read (Compliance Manager score, Insider Risk / Communication Compliance detail, DSPM), capture the figure from the portal via a named owner — and flag **"verify your licensing/tenant."** If Microsoft later ships a stable interface, switch that tile to `live` and reconcile it.
- **Show licensing state.** An E5-gated signal in an E3 tenant is a "not licensed" tile, not a blank. Buyers and auditors should see *why* a tile is empty.
- **Date every owner-captured figure.** A number with no "captured on" date is untrustworthy; validation flags stale figures so the board never quietly drifts.

Privacy & role separation (Insider Risk & Communication Compliance)

- **Aggregate only — always.** These HR-sensitive feeds appear as **counts** (policies active, open alerts), never as names, case content, or reviewable items. The dashboard must never become a back-door into case data.

- **Keep the collector out of investigator roles.** The account that builds the dashboard holds least-privilege **read** roles only and is **not** in Insider Risk / Communication Compliance investigator role groups. The program owners enter the aggregate figures themselves.
- **Distribute deliberately.** The rendered HTML is for the intended audience; do not scatter the raw dataset if any owner-captured figure is sensitive beyond that audience.

Audit

- **Enable ingestion once, deliberately.** If you enable unified audit-log ingestion, do it under change control (`-Commit`, piloted) — it is tenant-wide. Remember audit captures **from enablement forward**; set expectations that early dashboards have a short volume history.
- **Know Standard vs Premium.** Premium extends retention and adds record types; reflect which you have on the Audit tile so volume tiles are read in the right context.

Operations

- **Pilot, reconcile, then schedule.** Prove the numbers against the portal (Testing Trust gate) before anyone makes a decision from the board. Re-validate after every scheduled run and every added signal.
- **Schedule under a least-privilege identity, no secrets.** Run the collector as a service identity with only the read roles it needs; never embed credentials, tokens, or connection secrets in the scripts.
- **Version the config.** Keep `RiskComplianceDashboard.config.json` in source control (thresholds are policy) so changes to RAG bands or scope are reviewable and reversible.
- **Document rollback.** Deleting the local artifacts removes the dashboard with **no tenant change**; only audit-ingestion enablement is a tenant change — revert it only with compliance sign-off (disabling audit reduces evidence).

Automation & security

- **Read-only by default.** `CMPDASH-Discovery` / `CMPDASH-Scope` / `CMPDASH-Report` / `CMPDASH-Validation` are read-only; `CMPDASH-Set-Configuration` defaults to `-WhatIf` — preview before `-Commit`.
- **Least Graph scopes.** Connect with only the read scopes each run needs (e.g. `AuditLog.Read.All`); don't request write scopes for a reporting job.
- **Defer determinations.** The dashboard evidences posture; regulatory, records, and legal calls belong to the governance functions — keep that boundary explicit in how the board is presented.

8. Deliverables Summary — Risk & Compliance Dashboard (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The consolidated-dashboard model, the eight Purview signals, live vs owner-captured sources, privacy-by-design for HR-sensitive feeds
2. Requirements Guide	Every dashboard prerequisite with Verify / Implement-if-missing (category Purview GRs referenced, not duplicated)
Global Requirements (Purview)	Category-wide Purview prerequisites, published once
3. Implementation Guide	Exact, ordered S&C PowerShell + Graph + portal steps: discover signals, author config, pilot collect/render read-only, reconcile, schedule, distribute; `-Commit` write gated + rollback
5. Testing Guide	Validation procedures with pass/fail gates (readiness / build / trust)
6. Troubleshooting Guide	Signal, audit, reconciliation, privacy, and scheduling issues by symptom → root cause
7. Best Practices	Field-proven recommendations for a trustworthy, low-maintenance dashboard

PowerShell scripts (Security & Compliance PowerShell / Microsoft Graph)

Script	What it does	Safety
`CMPDASH-Discovery.ps1`	Signal-availability readiness: which of the eight Purview signals are present/enabled, E5 gating, DLP/label/retention policy counts, unified audit-log ingestion state	Read-only
`CMPDASH-Set-Configuration.ps1`	Writes `RiskComplianceDashboard.config.json` (tiles/RAG thresholds/cadence/sources) and a schedule scaffold; optionally enables audit-log ingestion	Default `-WhatIf`; `-Commit` to write
`CMPDASH-Validation.ps1`	Reconciles live tiles against the source (policy counts, DLP volume tolerance, audit state) and flags stale owner-captured figures	Read-only

Script	What it does	Safety
`CMPDASH-Report.ps1`	The consolidated dashboard generator: collects live signals + merges owner-captured figures → `RiskComplianceDashboard.data.json` and renders `RiskComplianceDashboard.html` + `.csv` with RAG + source badges	Read-only
`CMPDASH-Scope.ps1`	Scope/health helper: which feeds the current identity can read (role separation check) and each feed's freshness/health	Read-only

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

A **consolidated risk & compliance dashboard** across Microsoft Purview signals — Compliance Manager, DLP, sensitivity labels, retention/records, Insider Risk, Communication Compliance, Audit, and DSPM — rendered as an owned HTML/CSV artifact with RAG status and honest source badges, piloted read-only, reconciled against the portal, and refreshed on a schedule. HR-sensitive feeds are **aggregate-only and role-separated**, and all regulatory/records/legal determinations remain **deferred to the governance functions**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CMPDASH-Discovery.ps1

Purpose: Reports which of the consolidated dashboard's Purview signals are present/enabled in the tenant and which require Microsoft 365 E5 / E5 Compliance: DLP / sensitivity-label / retention policy counts (Security & Compliance PowerShell), the unified audit-log ingestion state, and a licensing note for the signals with no reliable public read (Compliance Manager, Insider Risk, Communication Compliance, DSPM) — which the dashboard treats as owner-captured. Read-only; makes no changes and reads no HR-sensitive case content. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CMPDASH-Discovery.ps1 Connect first: Connect-IPPSSession (ExchangeOnlineManagement) as a least-privilege reader (Compliance Administrator / Global Reader / View-Only Audit Logs). PowerShell 5.1+ or 7+. EXPECTED RESULTS Per signal: present/enabled or "not licensed / verify your tenant"; DLP/label/retention counts; audit ingestion on/off. Signals without a stable public read are listed as owner-captured. .NOTES Purview category — Risk & Compliance Dashboard. Read-only. Feature availability is licensing-dependent and the portal evolves — verify your licensing/tenant. Regulatory/records/legal determinations deferred. > [CmdletBinding()] param([switch]$IncludeAuditState = $true)

How to run: powershell -ExecutionPolicy Bypass -File .\CMPDASH-Discovery.ps1 Connect first: Connect-IPPSSession (ExchangeOnlineManagement) as a least-privilege reader (Compliance Administrator / Global Reader / View-Only Audit Logs). PowerShell 5.1+ or 7+. EXPECTED RESULTS Per signal: present/enabled or "not licensed / verify your tenant"; DLP/label/retention counts; audit ingestion on/off. Signals without a stable public read are listed as owner-captured. .NOTES Purview category — Risk & Compliance Dashboard. Read-only. Feature availability is licensing-dependent and the portal evolves — verify your licensing/tenant. Regulatory/records/legal determinations deferred. > [CmdletBinding()] param([switch]$IncludeAuditState = $true)

CMPDASH-Report.ps1

Purpose: The dashboard generator. Reads each 'live' Purview signal (DLP / sensitivity-label / retention policy enumeration via Security & Compliance PowerShell, and activity volume via Search-UnifiedAuditLog over the config window), merges the 'owner-captured' figures already recorded in RiskComplianceDashboard.config.json (Compliance Manager, Insider Risk, Communication Compliance, DSPM — aggregate-only), applies the RAG thresholds, writes the normalized dataset RiskComplianceDashboard.data.json, and renders RiskComplianceDashboard.html + .csv with one tile per signal (value + RAG colour + source badge). Read-only: reads posture, writes only local artifacts, changes no policy, and never reads HR-sensitive case content. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CMPDASH-Report.ps1 -WindowDays 30 Connect first: Connect-IPPSSession as a least-privilege reader (DLP/label/retention + audit read). PowerShell 5.1+ or 7+. EXPECTED RESULTS RiskComplianceDashboard.data.json, RiskComplianceDashboard.html, and RiskComplianceDashboard.csv written locally; unlicensed/unreadable signals shown as "not licensed / verify your tenant". .NOTES Purview category — Risk & Compliance Dashboard. Read-only. HR-sensitive feeds are aggregate-only, owner-captured. RAG thresholds are your policy. Determinations deferred to compliance/records/legal. > [CmdletBinding()] param([string]$ConfigPath = "./RiskComplianceDashboard.config.json", [string]$DataPath = "./RiskComplianceDashboard.data.json",

[string]$HtmlPath = "./RiskComplianceDashboard.html", [string]$CsvPath = "./RiskComplianceDashboard.csv", [int]$WindowDays = 30)

How to run: powershell -ExecutionPolicy Bypass -File .\CMPDASH-Report.ps1 -WindowDays 30 Connect first: Connect-IPPSSession as a least-privilege reader (DLP/label/retention + audit read). PowerShell 5.1+ or 7+. EXPECTED RESULTS RiskComplianceDashboard.data.json, RiskComplianceDashboard.html, and RiskComplianceDashboard.csv written locally; unlicensed/unreadable signals shown as "not licensed / verify your tenant". .NOTES Purview category — Risk & Compliance Dashboard. Read-only. HR-sensitive feeds are aggregate-only, owner-captured. RAG thresholds are your policy. Determinations deferred to compliance/records/legal. > [CmdletBinding()] param([string]$ConfigPath = "./RiskComplianceDashboard.config.json", [string]$DataPath = "./RiskComplianceDashboard.data.json", [string]$HtmlPath = "./RiskComplianceDashboard.html", [string]$CsvPath = "./RiskComplianceDashboard.csv", [int]$WindowDays = 30)

CMPDASH-Scope.ps1

Purpose: Preflights the Risk & Compliance Dashboard collector identity: for each live feed (DLP, sensitivity labels, retention, audit config) it reports read OK / permission-denied and a freshness/health note, so a role gap is not mistaken for a missing signal. It also PROVES role separation — the collector must NOT be able to read Insider Risk / Communication Compliance case content; those feeds must be aggregate/owner-captured. Read-only; reads no HR-sensitive case content. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CMPDASH-Scope.ps1 Connect first: Connect-IPPSSession (ExchangeOnlineManagement) as the least-privilege collector identity (Requirement 2.2). Optionally Connect-MgGraph for audit-read confirmation. PowerShell 5.1+/7+. EXPECTED RESULTS Per feed: readable / not-readable (role) + a health note; and confirmation that the HR-sensitive feeds are NOT readable by this account (role separation holds). Resolve any red before piloting. .NOTES Purview category — Risk & Compliance Dashboard. Read-only. Availability is licensing-dependent — verify your licensing/tenant. Insider Risk / Communication Compliance are aggregate/owner-captured by design. > [CmdletBinding()] param([switch]$SkipGraph)

How to run: powershell -ExecutionPolicy Bypass -File .\CMPDASH-Scope.ps1 Connect first: Connect-IPPSSession (ExchangeOnlineManagement) as the least-privilege collector identity (Requirement 2.2). Optionally Connect-MgGraph for audit-read confirmation. PowerShell 5.1+/7+. EXPECTED RESULTS Per feed: readable / not-readable (role) + a health note; and confirmation that the HR-sensitive feeds are NOT readable by this account (role separation holds). Resolve any red before piloting. .NOTES Purview category — Risk & Compliance Dashboard. Read-only. Availability is licensing-dependent — verify your licensing/tenant. Insider Risk / Communication Compliance are aggregate/owner-captured by design. > [CmdletBinding()] param([switch]$SkipGraph)

CMPDASH-Set-Configuration.ps1

Purpose: Writes RiskComplianceDashboard.config.json — the dashboard's tiles, RAG thresholds, refresh cadence, distribution list, and per-signal source (live vs owner-captured). Optionally enables unified audit-log ingestion (Set-AdminAuditLogConfig), which is tenant-wide and therefore gated. Safe by default: previews with -WhatIf and changes nothing until -Commit. Writes only local files plus, when requested, the one tenant setting (audit ingestion) — no Purview policy is changed. HOW TO RUN Preview : powershell -ExecutionPolicy Bypass -File .\CMPDASH-Set-Configuration.ps1 -WhatIf Write cfg: powershell -ExecutionPolicy Bypass -File .\CMPDASH-Set-Configuration.ps1 -Commit Enable audit ingestion (piloted, change control): .\CMPDASH-Set-Configuration.ps1 -EnableAuditIngestion -Commit Connect first:

Connect-IPPSSession as an admin with the required roles. PowerShell 5.1+ or 7+. EXPECTED RESULTS With -WhatIf: prints the config it would write and any action it would take, no changes. With -Commit: RiskComplianceDashboard.config.json written; audit ingestion enabled only if asked. .NOTES Purview category — Risk & Compliance Dashboard. Default -WhatIf. Enabling audit is high-impact — pilot + change control. RAG thresholds are your policy, not a Microsoft default. No secrets in this script. > [CmdletBinding()] param([string]$ConfigPath = "./RiskComplianceDashboard.config.json", [int]$WindowDays = 30, [string[]]$Distribution = @('compliance-leads@contoso.com'), [switch]$EnableAuditIngestion, [switch]$Commit, [switch]$WhatIf = $true)

How to run: Preview : powershell -ExecutionPolicy Bypass -File .\CMPDASH-Set-Configuration.ps1 -WhatIf Write cfg: powershell -ExecutionPolicy Bypass -File .\CMPDASH-Set-Configuration.ps1 -Commit Enable audit ingestion (piloted, change control): .\CMPDASH-Set-Configuration.ps1 -EnableAuditIngestion -Commit Connect first: Connect-IPPSSession as an admin with the required roles. PowerShell 5.1+ or 7+. EXPECTED RESULTS With -WhatIf: prints the config it would write and any action it would take, no changes. With -Commit: RiskComplianceDashboard.config.json written; audit ingestion enabled only if asked. .NOTES Purview category — Risk & Compliance Dashboard. Default -WhatIf. Enabling audit is high-impact — pilot + change control. RAG thresholds are your policy, not a Microsoft default. No secrets in this script. > [CmdletBinding()] param([string]$ConfigPath = "./RiskComplianceDashboard.config.json", [int]$WindowDays = 30, [string[]]$Distribution = @('compliance-leads@contoso.com'), [switch]$EnableAuditIngestion, [switch]$Commit, [switch]$WhatIf = $true)

CMPDASH-Validation.ps1

Purpose: The pilot gate for the Risk & Compliance Dashboard. Re-reads each live signal (DLP / sensitivity- label / retention policy counts and the unified audit-ingestion state) and reconciles it against the values persisted in RiskComplianceDashboard.data.json, and flags any owner-captured figure whose capturedOn date is older than the config's ownerCaptureMaxAgeDays. Read-only — makes no changes and reads no HR-sensitive case content; it only reports PASS/FAIL/STALE per tile. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CMPDASH-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\CMPDASH-Validation.ps1 -DataPath .\RiskComplianceDashboard.data.json -ConfigPath .\RiskComplianceDashboard.config.json Connect first: Connect-IPPSSession as the least-privilege reader. Run CMPDASH-Report.ps1 first so the dataset exists. PowerShell 5.1+ or 7+. EXPECTED RESULTS A PASS/FAIL line per live tile (live re-read matches the dataset) and PASS/STALE per owner-captured tile. All PASS = the dashboard reconciles to source (pilot gate met). Any FAIL/STALE = refresh the report or re-capture the owner figure before scheduling/distributing. .NOTES Purview category — Risk & Compliance Dashboard. Read-only. Confirms the dashboard read source correctly, NOT whether the underlying posture satisfies a regulation (a compliance/records/legal determination). > [CmdletBinding()] param([string]$DataPath = "./RiskComplianceDashboard.data.json", [string]$ConfigPath = "./RiskComplianceDashboard.config.json")

How to run: powershell -ExecutionPolicy Bypass -File .\CMPDASH-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\CMPDASH-Validation.ps1 -DataPath .\RiskComplianceDashboard.data.json -ConfigPath .\RiskComplianceDashboard.config.json Connect first: Connect-IPPSSession as the least-privilege reader. Run CMPDASH-Report.ps1 first so the dataset exists. PowerShell 5.1+ or 7+. EXPECTED RESULTS A PASS/FAIL line per live tile (live re-read matches the dataset) and PASS/STALE per owner-captured tile. All PASS = the dashboard reconciles to source (pilot gate met). Any FAIL/STALE = refresh the report or re-capture the owner figure before scheduling/distributing. .NOTES Purview category — Risk & Compliance Dashboard. Read-only. Confirms the dashboard read source correctly, NOT whether the underlying posture satisfies a regulation (a

compliance/records/legal determination). > [CmdletBinding()] param([string]$DataPath = "./RiskComplianceDashboard.data.json", [string]$ConfigPath = "./RiskComplianceDashboard.config.json")